

Project Design Phase
Proposed Solution Template

Date	30 September 2026
Team ID	SWUID2026286057
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Marks

Proposed Solution Template:

Project team shall fill the following information in the proposed solution template.

S.No.	Parameter	Description
1.	Problem Statement (Problem to be solved)	In the default ServiceNow Incident form every field is shown and accepted exactly as typed, with no contextual rules. Because the form does not guide the user, the following problems occur: • Manual entry errors: Caller, Category, Subcategory, Impact and Urgency are filled in by hand and inconsistently. A wrong category or vague impact leads to mis-routing to the wrong assignment group. • Incomplete incident submissions: Key details such as Short description, Description and Contact type can be skipped, and incidents can be moved to Resolved without Close code or Close notes, leaving no resolution history. • Improper field validation: There are no checks for meaningless one-word descriptions, illogical combinations (e.g. a State set to On Hold without a reason), or edits to fields after an incident is closed. • Cluttered, non-contextual forms: Fields irrelevant to the current State are always visible, which confuses both end users and service desk agents. • Slow resolution times: Agents must keep going back to callers for missing information, reassign tickets and reopen incomplete ones. This increases Mean Time to Resolve (MTTR), causes SLA breaches and reduces the quality of reporting data. A lightweight, browser-side mechanism is therefore needed to validate data and control form behaviour at the moment the user is working on the form.
2.	Idea / Solution description	The solution configures Client Scripts and UI Policies on the Incident [incident] table so that the form behaves dynamically and validates input before it reaches the server. A. Client Scripts (JavaScript, runs in the browser) • onLoad: Runs when the form opens. Sets default values (e.g. Contact type), shows a guidance message to new-incident users, and locks fields when the incident is Closed. • onChange: Runs when a field value changes. Examples: on Category change, clear Subcategory and show a hint; on Caller change, auto-populate caller details (location, department) using

		<code>g_form.getReference()</code> and alert agents for VIP callers; on State change, toggle resolution and hold fields. • onSubmit: Runs when Save/Submit is clicked. Performs final validation (minimum Short description length, Close code and Close notes when Resolved, reason when On Hold) and stops submission with an error message if a rule fails. B. UI Policies (declarative, no-code/low-code conditions) • Mandatory: Short description, Category, Impact and Urgency are always mandatory; Close code and Close notes become mandatory when State is Resolved; On hold reason becomes mandatory when State is On Hold. • Read-only: Incident Number, Caller (after creation) and calculated Priority are read-only; all key fields become read-only once the incident is Closed. • Visible: Resolution fields appear only for Resolved/Closed states, and the On hold reason field appears only when State is On Hold, which keeps the form clean. C. Implementation approach • Create the rules in the Incident table using Client Scripts and UI Policies, with clear names, conditions and execution order. • Use “Reverse if false” in UI Policies so that fields return to their normal state automatically when the condition no longer applies. • Test each rule with different user roles (caller, ITIL agent, admin) using impersonation, then move the changes between instances using an Update Set.
3.	Novelty / Uniqueness	• Contextual validation rules: Validation depends on the State, Category, Impact and user role instead of a fixed, one-size-fits-all form. • Dynamic field toggling without server-side load: Fields are shown, hidden, made mandatory or read-only directly in the browser, so there is no page reload and no extra Business Rule execution on the server. • Improved agent experience (UX): Agents see only the fields relevant to the current stage, and caller details are auto-populated so less typing is needed. • Real-time guidance: Field messages, info messages and error messages guide users while they type instead of rejecting the record after submission. • Layered design: UI Policies handle simple conditions declaratively, while Client Scripts handle complex logic. This makes the solution easy to maintain, upgrade-safe and free of external plugins or licences.
4.	Social Impact / Customer Satisfaction	• Faster incident resolution (MTTR reduction): Complete and correct information at the time of logging removes follow-up questions and reassignments, so incidents reach the right team and are resolved sooner. • Clean, reliable ticket data: Mandatory and validated fields improve data quality for reports, dashboards, trend analysis and Knowledge Base creation from resolved incidents. • Smoother end-user and service desk interaction: Clear guidance and a simple, uncluttered form reduce confusion for end users and

		reduce the workload and stress on agents. • Higher customer satisfaction (CSAT): Quicker response and resolution, fewer repeat questions and more predictable service improve user trust and CSAT scores, and reduce SLA breaches. • Wider benefit: Employees lose less working time because of IT issues, which supports overall organisational productivity.
5.	Business Model (Revenue Model)	This is an internal ITSM improvement, so the model is based on value addition and cost savings rather than direct sales revenue. • Operational efficiency: Less time spent per ticket on corrections, clarifications and rework, so the same service desk team can handle more incidents. • Reduced processing overhead: Fewer reassignments, reopened tickets and escalations, which lowers the cost per ticket. • Automated governance: Data standards and process rules are enforced automatically by the form instead of by manual audits and supervisor checks, which supports ITIL and audit compliance. • Low implementation cost: Built with standard ServiceNow features (no extra licences, plugins or custom applications), with low maintenance cost and a quick return on investment. • Reusable asset: The rule set can be packaged as a standard configuration template that IT service providers or partners can reuse for different clients.
6.	Scalability of the Solution	• Across ITIL modules: The same pattern (onLoad/onChange/onSubmit Client Scripts plus Mandatory/Read-only/Visible UI Policies) can be applied to Problem Management, Change Management and Request/Task-based forms with only field names and conditions changed. • Service Catalog: Catalog Client Scripts and Catalog UI Policies apply the same logic to catalog item variables, giving consistent validation in the Service Portal. • Client-side performance: Rules execute in the user's browser, so load on the server does not grow with the number of users or incidents. • Reusable and maintainable: Common logic can be kept in reusable scripts and standard naming conventions, and moved between Dev, Test and Production using Update Sets or a scoped application. • Flexible rollout: Rules can be restricted by role, group or condition, and extended step by step as new requirements or departments are added.